

# Nanjing University of Information Science and Technology

## Laboratory Report

### Experiment Title: Containerization with Docker

|                |                                 |            |                                           |
|----------------|---------------------------------|------------|-------------------------------------------|
| Date           | 2026-06-10                      | Class      | 2                                         |
| Name           | 姜丞骏                             | Student ID |                                           |
| GitHub Account | whatokk                         | Repository | whatokk/lab7-containerization-with-docker |
| Environment    | GitHub Actions<br>Ubuntu runner | Image      | cloud-lab-image:v1                        |

#### 一、Experimental Purpose

- Learn how to package a web application into a Docker image.
- Understand immutable infrastructure by comparing source-code edits with the running container snapshot.
- Practice network port mapping between host ports and container port 80.
- Explore container isolation, resource monitoring, scaling, port conflicts, and cleanup.

#### 二、Experiment Contents

- Verified Docker client and server versions.
- Created a Dockerfile based on nginx:alpine.
- Built cloud-lab-image:v1.
- Ran the container on host port 8080 and accessed the web page.
- Edited source HTML and confirmed the running container did not change.
- Used docker exec to inspect files inside the container.
- Used docker stats to monitor CPU and memory usage.
- Started a second instance on port 8081 and reproduced the 8080 port conflict.
- Stopped and removed containers, then verified cleanup with docker ps.
- Saved all source files and evidence in GitHub.

## 三、 Detailed Steps and Results

### Step 1 Verify Docker Environment

Docker Engine client and server were available in the GitHub Actions Ubuntu environment. Both reported Docker Engine Community version 28.0.4.

### Step 2 Create Dockerfile

Created a Dockerfile using nginx:alpine, copied index.html into /usr/share/nginx/html/index.html, and exposed container port 80.

### Step 3 Build Docker Image

Ran `docker build -t cloud-lab-image:v1 .` and created an image of 62.3MB.

### Step 4 Run Container

Ran `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1`. The host port 8080 mapped to container port 80.

### Step 5 Access Web Server

Used `curl http://127.0.0.1:8080` and received the personal HTML page served from inside the Nginx container.

### Step 6 Test Immutability

Edited source index.html after the container was running. The web response did not change until rebuild/restart, proving the running container is based on an immutable image snapshot.

### Step 7 Explore Container Interior

Used `docker exec` to list `/usr/share/nginx/html/` and read the container copy of index.html. This showed the container has its own isolated filesystem.

### Step 8 Resource Monitoring

`docker stats --no-stream` showed my-web-container using 0.00% CPU and 10.39MiB memory, much lighter than a traditional VM.

### Step 9 Scaling and Port Management

Started second-container on host port 8081. Trying to start third-container on host port 8080 produced `Bind for 0.0.0.0:8080 failed: port is already allocated`.

### Step 10 Cleanup

Stopped and removed the containers. `docker ps` showed no running containers afterwards.

### Step 11 Git Save

Committed the Dockerfile, index.html, workflow, and report evidence to GitHub.

### 四、Evidence Screenshots

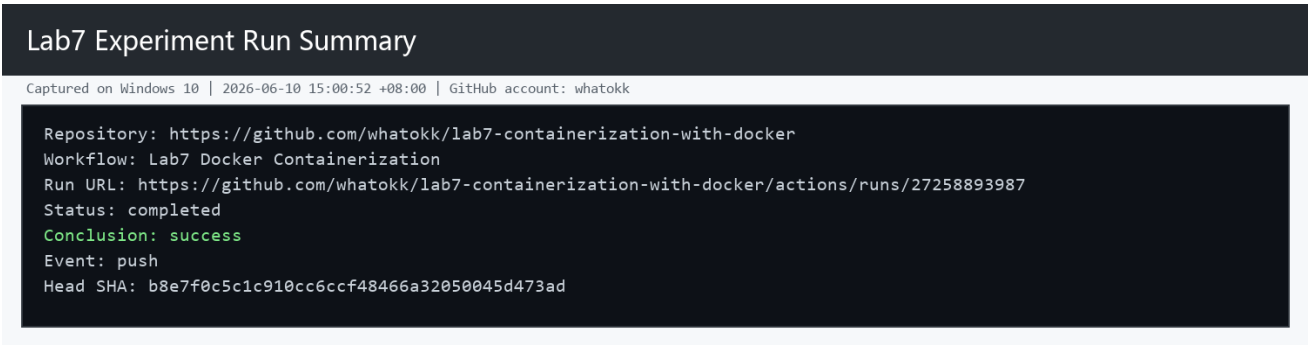


Figure 1. Lab7 GitHub Actions run summary.

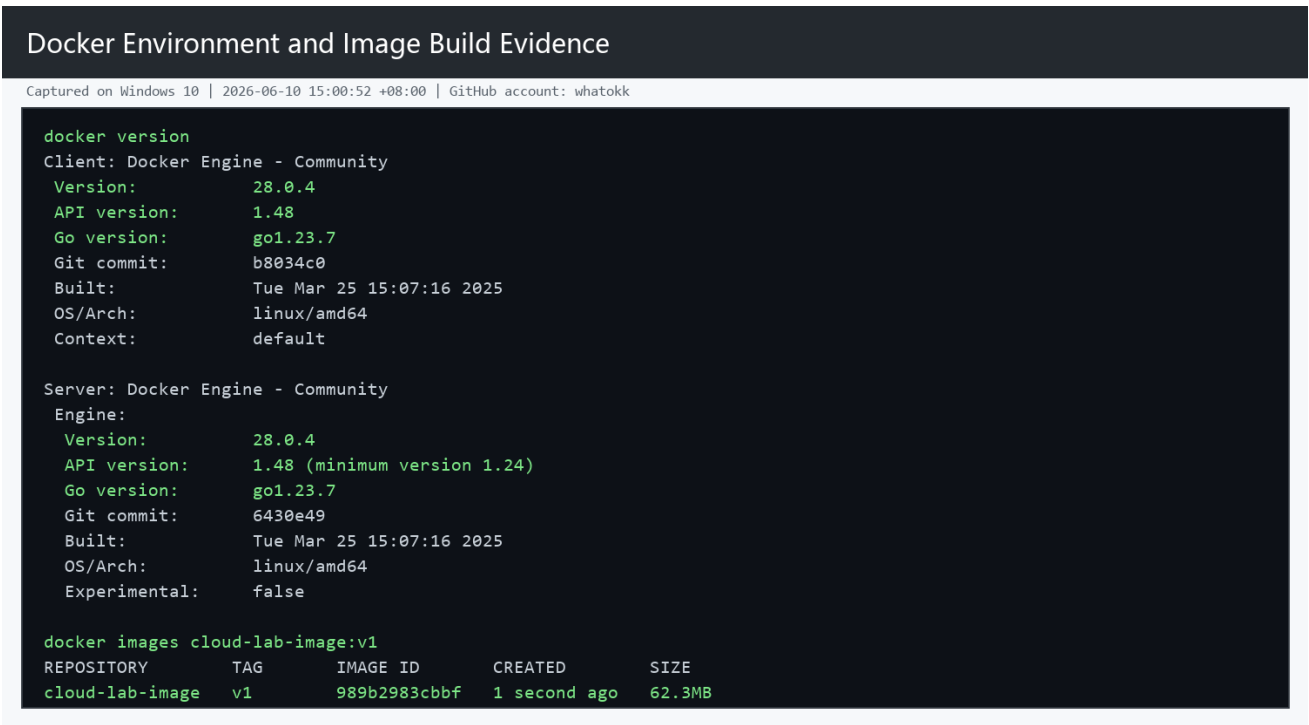


Figure 2. Docker environment and image build result.

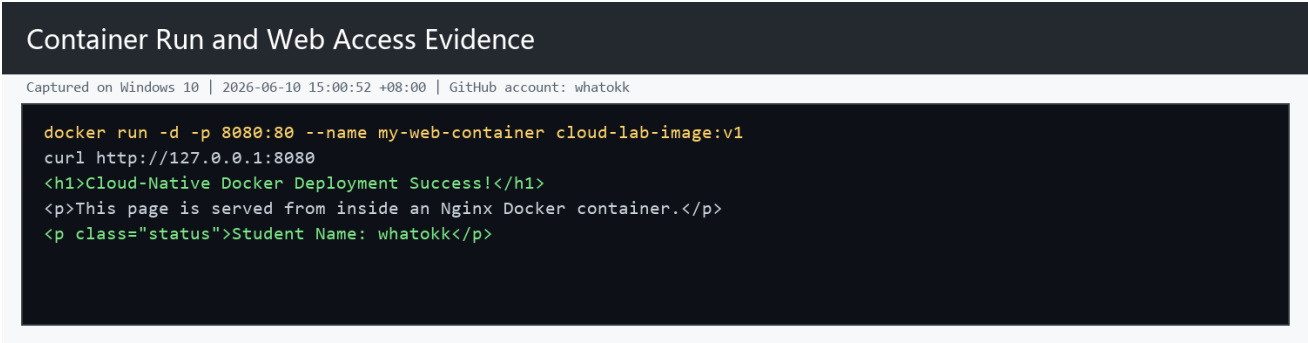


Figure 3. Container run and web server access result.

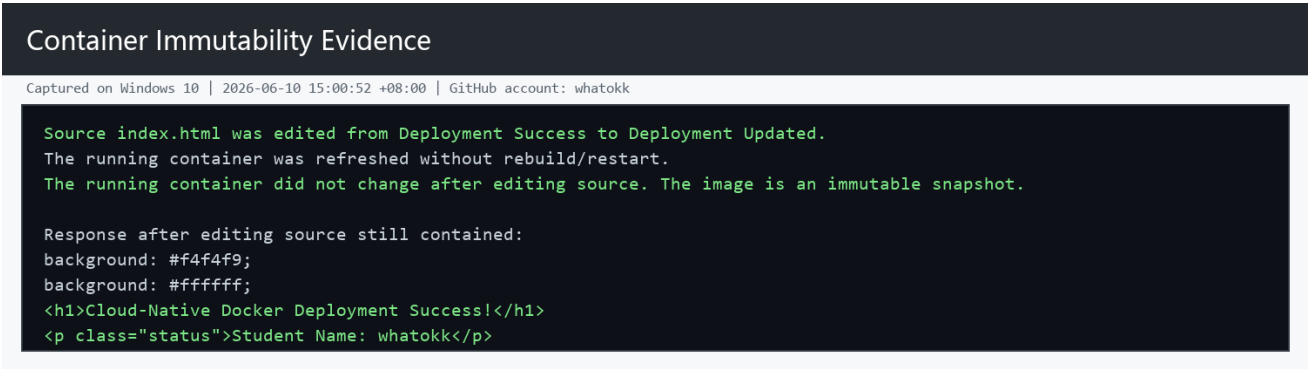


Figure 4. Container immutability test result.

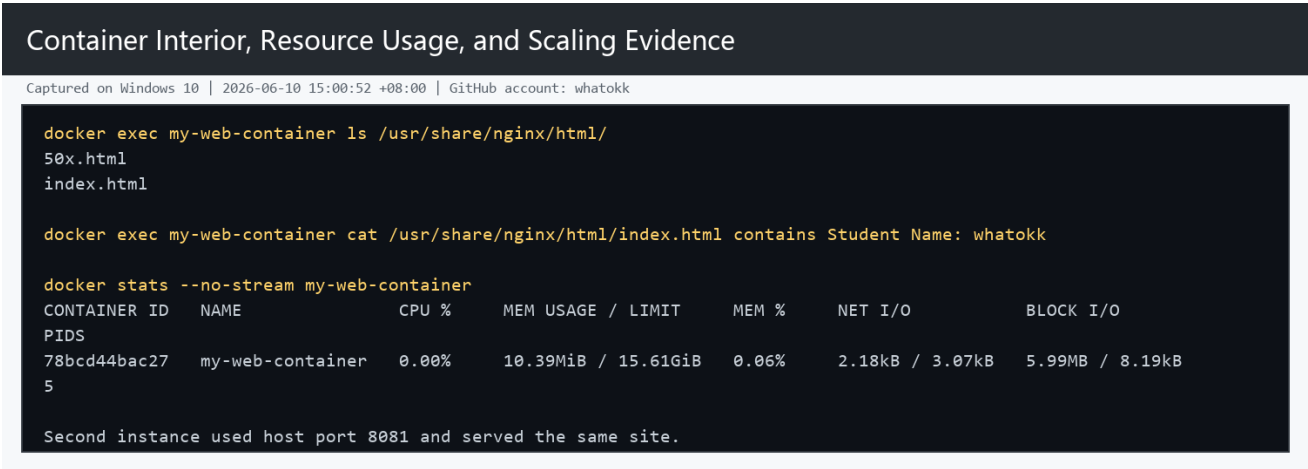


Figure 5. Container filesystem, resource usage, and scaling result.

Port Conflict and Cleanup Evidence

Captured on Windows 10 | 2026-06-10 15:00:52 +08:00 | GitHub account: whatokk

```
docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint third-container (55da9e1c2c6885ecc932732e1aa9e58463983068fd4e329d6554593e43797daa): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information

After cleanup:
```

| CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS | PORTS | NAMES |
|--------------|-------|---------|---------|--------|-------|-------|
|--------------|-------|---------|---------|--------|-------|-------|

Figure 6. Port conflict and cleanup result.

五、Reflection

Problems Encountered and Proposed Solutions: The local Windows Docker daemon did not return stable server information, so the Docker experiment was executed on a GitHub Actions Ubuntu runner. This still matches the cloud-computing purpose because the commands ran on an on-demand cloud VM and produced reproducible logs and artifacts.

Why did the website not update automatically after editing the code? The running container was created from the previously built image, which is a static snapshot. Editing the source file outside the container does not alter the filesystem already packaged into the image. This is a cloud feature because immutable images make deployments repeatable, auditable, and easy to roll back. To apply changes, engineers rebuild the image and restart or redeploy containers.

Port and scaling reflection: Multiple identical containers can run at the same time, but each host port can only be bound once. The second instance succeeded on 8081, while a third container failed on 8080 because that port was already used by my-web-container.

六、Links

Experiment repository: <https://github.com/whatokk/lab7-containerization-with-docker>

Successful Actions run: <https://github.com/whatokk/lab7-containerization-with-docker/actions/runs/27258893987>